

Taller 3 — Aprendizaje No Supervisado · Investigación Operativa · Algoritmos Genéticos

Maestría en Inteligencia Artificial – USFQ

Oldrin BONILLA

Stan MORA

Tais RODRIGUEZ

El presente documento consolida el modelado, análisis e implementación de distintos problemas clásicos de inteligencia artificial mediante técnicas de búsqueda, representación de estados, optimización y visualización interactiva, integrando herramientas modernas de desarrollo bajo un enfoque de low-code engineering.

Introducción	4
Distribución de contribuciones	5
P1 — Aprendizaje No Supervisado	5
P2 — Travelling Salesman Problem (TSP)	9
P3 — Algoritmos Genéticos	15
Conclusión General	23

Introducción

El presente taller aborda el análisis y la resolución de distintos problemas clásicos de inteligencia artificial mediante un enfoque orientado al modelado computacional, la representación del conocimiento y la exploración de espacios de estados. A partir de fundamentos asociados a teoría de grafos, algoritmos de búsqueda, razonamiento lógico y optimización combinatoria, se desarrollaron soluciones capaces de representar procesos de decisión y construir estrategias de resolución para problemas de distinta complejidad computacional.

Los casos abordados —el Problema del Viajante (Travelling Salesman Problem), el Problema del Granjero y las Torres de Hanoi— constituyen referencias fundamentales dentro del estudio de la inteligencia artificial y las ciencias de la computación, debido a que permiten analizar diferentes mecanismos de búsqueda, representación de estados, optimización y planificación secuencial. Cada problema fue desarrollado no únicamente desde una perspectiva algorítmica, sino también como un ejercicio de abstracción y modelado formal orientado a comprender la lógica subyacente de los procesos de resolución.

De manera complementaria, el proyecto incorporó mecanismos de visualización interactiva y simulación dinámica con el propósito de representar gráficamente la evolución de los estados, las transiciones, las decisiones del agente y el comportamiento de los algoritmos durante su ejecución. Este enfoque permitió fortalecer la interpretación conceptual de las soluciones implementadas y facilitar el análisis del comportamiento computacional de cada problema.

El desarrollo fue realizado mediante tecnologías como Python, Flask, Tkinter y herramientas web de visualización, integradas bajo un enfoque de low-code engineering apoyado por herramientas de inteligencia artificial para optimizar tareas de programación, documentación y depuración. Asimismo, durante todas las fases del taller se mantuvo documentación continua en Notion para el seguimiento conceptual, metodológico y operativo del proyecto, consolidándose finalmente el contenido técnico en el presente documento formal.

Distribución de contribuciones

Integrante	Contribución principal
Oldrin Bonilla	P1 — Aprendizaje No Supervisado (análisis, clustering y anomalías)
Stan Mora	P3 — Algoritmos Genéticos (implementación, casos de estudio y análisis)
Tais Rodríguez	P2 — Travelling Salesman Problem (TSP), documentación y redacción técnica

P1 — Aprendizaje No Supervisado

El aprendizaje no supervisado es una rama fundamental de la inteligencia artificial que se enfoca en identificar patrones y estructuras ocultas en conjuntos de datos sin la necesidad de etiquetas o supervisión externa. Para este taller se utilizó el Student Productivity & Behavior Dataset (SP&BDS), un conjunto de datos con variables de comportamiento estudiantil incluyendo horas de estudio, uso del celular, horas de sueño y puntuación de productividad.

Visualización de variables

Los resultados revelan la naturaleza estructural de los datos. En las estadísticas descriptivas, casi todas las variables de comportamiento como horas de estudio o uso del celular tienen una distribución completamente plana, es decir, hay la misma cantidad de estudiantes en cada extremo. Sin embargo, la puntuación de productividad tiene forma de campana normal, lo que sugiere que es el resultado calculado de las demás variables.

El gráfico de líneas entrelazadas evidencia una estructura altamente dispersa y sin patrones claramente diferenciables entre perfiles de productividad. Este comportamiento es consistente con el análisis PCA, el cual sugiere una baja presencia de factores latentes dominantes y una relativa independencia entre las variables observadas. En conjunto, estos resultados indican que el dataset presenta características compatibles con una generación sintética o pseudoaleatoria.

A pesar de esa dispersión, el algoritmo matemático logró identificar que la productividad y las horas de estudio son las variables de mayor peso. Al cruzarlas en

el gráfico final, se observa una tendencia realista: dedicar más horas al estudio tiende a elevar la productividad, pero la relación no es perfecta. Hay una amplia variación que demuestra que estudiar muchas horas ayuda, pero no garantiza el éxito por sí solo.

Encontrar patrones/clústeres — análisis univariable

El proceso comienza con la carga y limpieza habitual de los datos, asegurando que solo se analice información numérica válida. Dado que el conjunto de datos tiene muchas variables, el código utiliza la técnica de reducción de dimensiones PCA para comprimir toda esa información en solo dos componentes principales. Esto es vital porque permite que el agrupamiento sea más rápido y que los resultados sean visualizables en dos dimensiones.

Una vez comprimidos los datos, el algoritmo evalúa desde dos hasta ocho grupos mediante dos métricas clásicas: el método del codo y el coeficiente de silueta. Finalmente, selecciona automáticamente el número ideal de grupos basándose en el mejor puntaje y clasifica a cada estudiante.

Evaluación del número óptimo de grupos

La curva del método del codo desciende de forma constante sin formar un doblez claro, lo cual ocurre cuando los datos no tienen separaciones naturales obvias. El coeficiente de silueta muestra su pico más alto en dos grupos, por lo que el algoritmo decide dividir a los estudiantes en dos grandes perfiles. El puntaje máximo es relativamente bajo, lo que advierte que estos dos grupos se parecen mucho entre sí y no están fuertemente aislados.

El Mapa de Clústeres

El mapa de clústeres revela una distribución densamente concentrada y sin separaciones estructurales claramente definidas entre grupos. La ausencia de fronteras naturales o regiones vacías sugiere que el algoritmo realiza una partición forzada del espacio de datos más que una identificación de agrupamientos inherentes. Este comportamiento refuerza la hipótesis de que el conjunto analizado posee una distribución predominantemente uniforme y de naturaleza sintética.

Encontrar anomalías — análisis univariable

El algoritmo calcula el promedio general de cada variable y mide qué tan lejos está cada estudiante de ese centro. Se establece un límite de tolerancia matemático basado en Z-Score; si un estudiante supera ese límite, se le etiqueta como anomalía. Los gráficos de barras muestran la distribución y pintan de rojo las zonas extremas. Al analizar las variables de horas de estudio, uso del celular y horas de sueño, vemos que las barras forman bloques rectos y cuadrados — las fronteras punteadas caen completamente fuera de donde existen datos, por lo que evaluando estas variables individualmente no hay ninguna anomalía.

Por el contrario, la Puntuación de Productividad tiene forma de campana, acumulando la gran mayoría en el centro. Aquí las fronteras sí logran atrapar datos en los extremos: un grupo muy reducido con productividad críticamente baja y otro con desempeño excepcionalmente alto.

Encontrar patrones — análisis multivariable

El proceso toma a los estudiantes, los evalúa usando dos técnicas (K-Means y Clustering Jerárquico) pidiendo tres grupos, y luego cruza los resultados para calcular un índice de consistencia. Finalmente, busca el cruce con mayor cantidad de coincidencias para definir el patrón más representativo del conjunto.

Interpretación de las gráficas

En el gráfico K-Means el algoritmo logró forzar una división en tres grandes bloques de colores relativamente ordenados. Sin embargo, en el Clustering Jerárquico los colores están profundamente mezclados, superpuestos y caóticos.

La divergencia entre ambas técnicas de agrupamiento evidencia una baja estabilidad estructural en la organización de los datos. Mientras K-Means impone particiones relativamente ordenadas debido a su naturaleza centroidal, el Clustering Jerárquico refleja una distribución considerablemente más dispersa y superpuesta. En escenarios con patrones intrínsecos robustos, ambas metodologías tienden a producir estructuras comparables; la diferencia observada refuerza nuevamente la hipótesis de una distribución altamente aleatoria o sintética.

Encontrar anomalías — análisis multivariable

El análisis univariable confirma los resultados previos: horas de estudio, uso del celular y horas de sueño no presentan anomalías individuales, mientras que la

productividad muestra pequeñas anomalías en los extremos de su distribución normal.

Análisis Bivariable con Isolation Forest

El segundo bloque realiza una búsqueda de anomalías mucho más sofisticada al analizar dos características al mismo tiempo mediante Isolation Forest. Este sistema busca anomalías en la relación entre variables: estudiar poco es normal, tener baja productividad también, pero estudiar diez horas diarias y tener la productividad por los suelos es una combinación inusual.

En el cruce de horas de estudio versus productividad, la gran nube azul muestra una tendencia lógica ascendente. Los puntos rojos en los bordes señalan dos perfiles atípicos importantes: en la parte superior izquierda, estudiantes altamente eficientes que logran grandes resultados con mínimo esfuerzo (posiblemente con conocimientos previos sólidos); en la parte inferior derecha, estudiantes que dedican muchas horas pero tienen productividad bajísima, señal de agotamiento o malas técnicas de estudio.

El gráfico que cruza uso del celular con horas de sueño muestra una masa azul rectangular perfecta, indicando que no existe correlación real entre ambas variables en esta muestra. Los puntos rojos en las esquinas representan escenarios físicamente improbables, confirmando que el dataset fue generado sintéticamente.

Conclusiones

Análisis Univariable

Las variables de comportamiento tienen distribución uniforme plana mientras que la productividad tiene distribución normal, sugiriendo que es el resultado combinado de las demás. Las anomalías univariadas son únicamente los casos más extremos de productividad.

Análisis Bivariable: Horas de Estudio vs. Productividad

Existe una clara tendencia positiva confirmada. Las anomalías detectadas por Isolation Forest revelan dos perfiles educativamente significativos: alta eficiencia (pocas horas, alta productividad) e ineficiencia/burnout (muchas horas, baja productividad). Estos perfiles permiten intervención educativa temprana.

Análisis Bivariable: Uso del Celular vs. Horas de Sueño

La forma rectangular perfecta de la nube de datos confirma que no existe correlación y que el dataset fue generado sintéticamente. Las anomalías en las esquinas representan comportamientos físicamente imposibles en la vida real.

Conclusión General

Las anomalías multivariantes son mucho más valiosas que las univariantes. Mientras que el análisis univariante solo indica que un estudiante estudia 10 horas (lo cual no es malo por sí solo), el análisis multivariante permite identificar al estudiante que estudia 10 horas y está fracasando, posibilitando una intervención educativa temprana.

P2 — Travelling Salesman Problem (TSP)

El Travelling Salesman Problem (TSP) es uno de los problemas clásicos de la teoría de la complejidad computacional, clasificado como NP-duro. El objetivo es encontrar el ciclo hamiltoniano de menor longitud, es decir, la ruta más corta posible que visite todas las ciudades exactamente una vez y regrese al punto inicial. En este taller se implementa un modelo MILP usando Pyomo con el solver GLPK, comparando los resultados contra la heurística greedy del vecino cercano y evaluando dos heurísticas adicionales. Los resultados se visualizan mediante `tsp_viewer.py`, una interfaz gráfica interactiva que permite seleccionar cada caso, ajustar parámetros y comparar rutas en tiempo real.

Referencia: Vecino Cercano — 100 ciudades

La heurística del vecino cercano construye la ruta escogiendo siempre la ciudad no visitada más cercana a la posición actual. Con `seed=123` y 100 ciudades, obtuvo una distancia de 2006.73, coincidiendo exactamente con la referencia del enunciado. Esta solución sirve de baseline para evaluar si el LP con o sin heurísticas puede superarla.

Análisis del código propuesto — Caso 1

El modelo LP/MILP usa variables binarias $x[i,j]$ que indican si se viaja de ciudad i a ciudad j , restricciones de entrada/salida que garantizan exactamente un arco por

ciudad, y restricciones MTZ para eliminar subtours. Se corrió con mipgap=0.05 y límite de 30 segundos para 10, 20, 30, 40 y 50 ciudades.

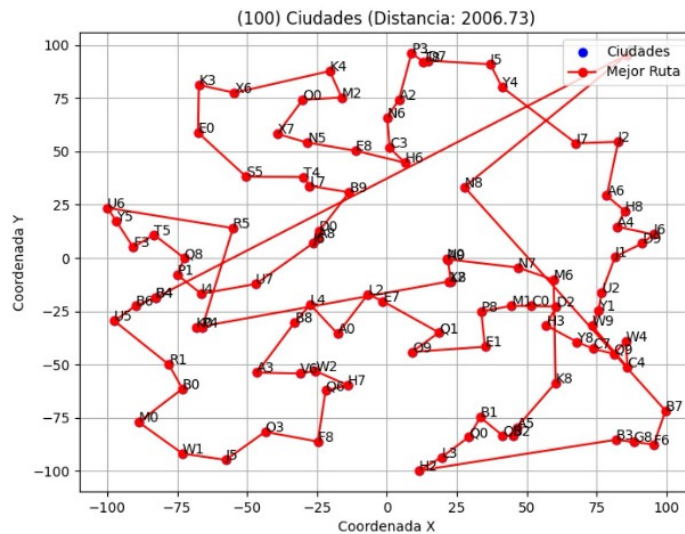


Figura 1. Ruta del vecino cercano — 100 ciudades (Distancia: 2006.73). Se observan múltiples cruces de caminos, característica de las soluciones greedy.

Ciudades	Tiempo LP	Dist. LP	Dist. Vecino	Mejora LP
10	< 1s	570.70	588.52	3.0%
20	~1s	717.85	758.23	5.3%
30	30s	853.44	1036.63	17.7%
40	30s	1085.32	1174.04	7.6%
50	30s	1096.57	1368.73	19.9%

Tabla 1. Comparación tiempos y distancias LP vs Vecino Cercano. ⌚ indica tiempo límite agotado.

Con pocas ciudades (10-20) el LP encuentra soluciones de alta calidad en menos de 2 segundos. A partir de 30 ciudades el solver siempre agota los 30 segundos, confirmando la naturaleza NP-duro del TSP. Aun así, el LP supera al vecino cercano entre un 3% y 20% incluso con tiempo agotado, porque el branch-and-bound explora el espacio de forma sistemática mientras que el vecino cercano toma decisiones irrevocables que acumulan errores visibles como cruces de caminos.

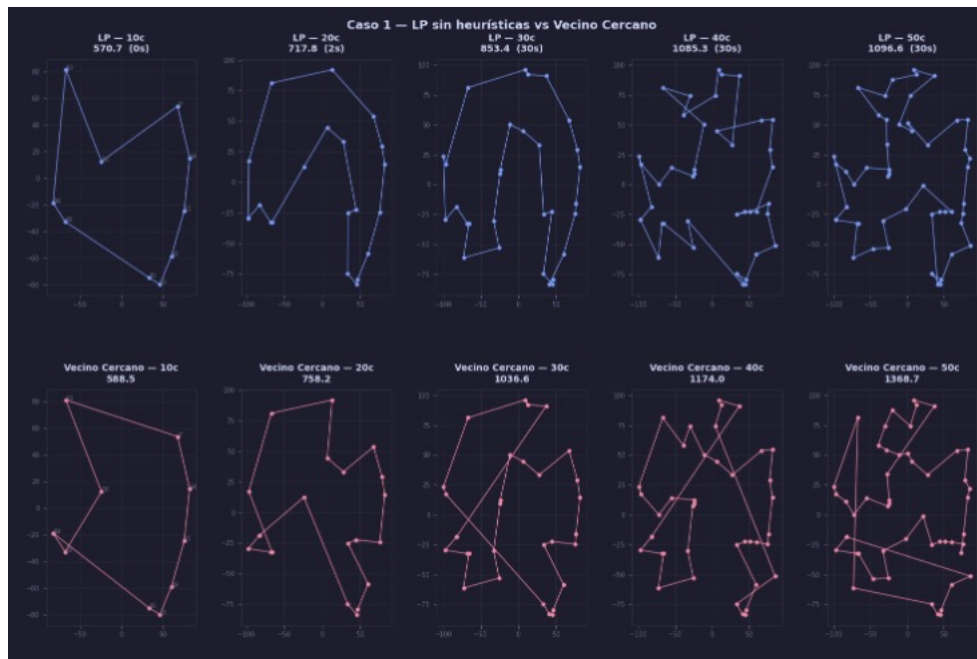


Figura 2. Vista del `tsp_viewer` — Caso 1: LP sin heurísticas (azul, fila superior) vs. Vecino Cercano (rosa, fila inferior) para 10 a 50 ciudades. El LP produce rutas más cortas y con menos cruces en todos los tamaños evaluados.

Parámetro `tee` — Funcionalidad y análisis

El parámetro `tee` controla si el solver imprime su progreso en pantalla. Con `tee=True`, GLPK muestra en tiempo real la mejor solución entera encontrada (cota superior mip), la cota inferior teórica (relajación LP) y el gap entre ambas. El solver acepta la solución cuando $\text{gap} \leq \text{mipgap}$ especificado. Este parámetro es fundamental para diagnosticar si el solver converge o está atascado — en el Caso 2 permitió observar que con heurística de límites el solver encontró su primera solución entera en la iteración 8,277 vs 21,018 sin heurística, evidencia directa del impacto de la heurística.

Heurística de límites a la función objetivo — Caso 2 (70 ciudades)

Esta heurística agrega dos restricciones que acotan la función objetivo entre un mínimo y máximo estimados: $\text{obj} \geq \text{medium_low} \times n \times 0.25$ y $\text{obj} \leq \text{medium_low} \times n \times 0.60$, donde $\text{medium_low} = (\text{dist_mínima} + \text{dist_promedio}) / 2$. Esto permite al

solver descartar ramas del árbol B&B que producirían rutas fuera del rango estimado.

Métrica	Con heurística	Sin heurística
Distancia total	1473.05	1651.85
Gap final	31.7%	39.2%
Primera solución entera (ite)	8,277	21,018
Mejora	10.8% mejor	baseline

Tabla 2. Resultados Caso 2 — con y sin heurística de límites (70 ciudades, mipgap=0.2, límite=40s).

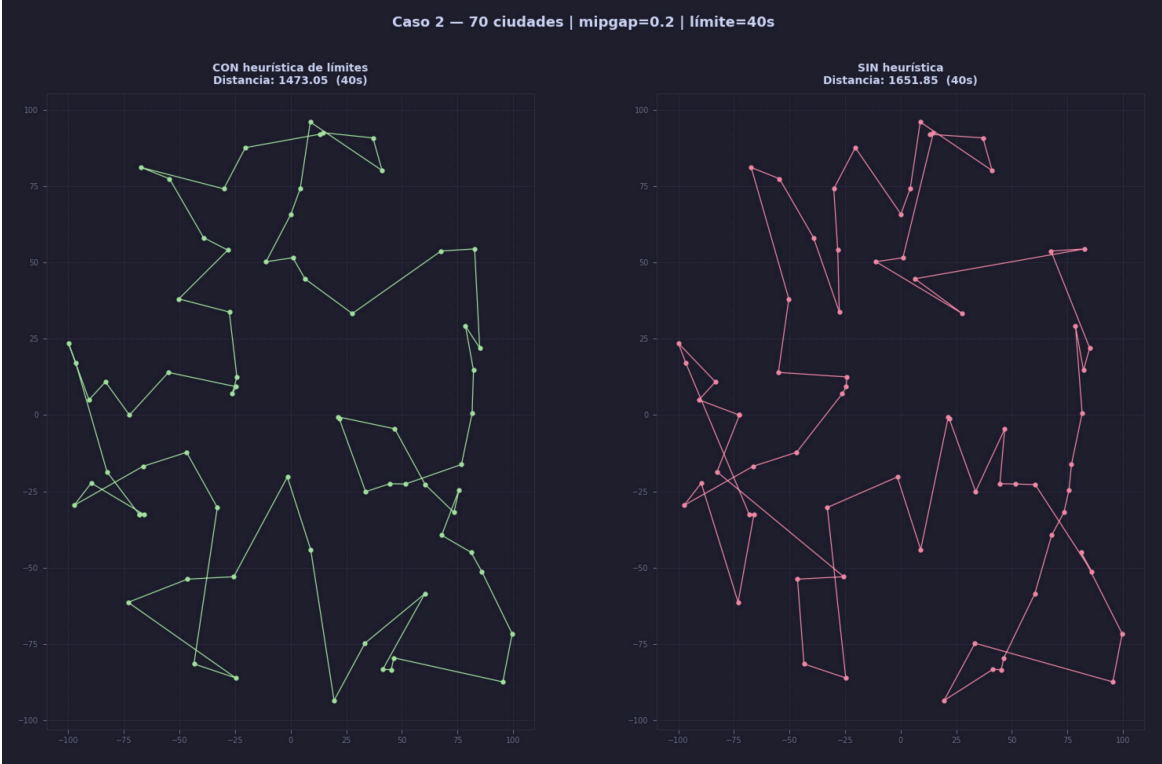


Figura 3. Vista del tsp_viewer — Caso 2: CON heurística de límites (verde, 1473.05) vs SIN heurística (rosa, 1651.85). La heurística logra una mejora del 10.8% en el mismo tiempo de cómputo.

¿Cuál es la diferencia entre los dos casos?

Con la heurística de límites el solver encontró su primera solución entera válida mucho antes y la refinó hasta 1473.05, un 10.8% mejor. El gap final también fue

menor (31.7% vs 39.2%), confirmando que la heurística orientó mejor la búsqueda al descartar automáticamente ramas que producirían rutas fuera del rango estimado [887.24, 2129.37].

¿Sirve esta heurística para cualquier caso?

No necesariamente. El rango se calcula con factores fijos (0.25 y 0.60) sobre la distancia media-baja. Si la distribución de ciudades fuera muy irregular, el rango podría ser incorrecto: si max_posible es demasiado bajo el problema se vuelve infactible; si min_posible es demasiado alto se excluye la solución óptima real. Funciona bien con distribuciones uniformes aleatorias pero puede fallar con clusters o geometrías inusuales.

Heurística de vecinos cercanos en LP — Caso 3 (100 ciudades)

Esta heurística agrega restricciones que limitan qué arcos $x[i,j]$ son válidos para ciudades con muchos vecinos cercanos, reduciendo el problema de 10,000 a 5,922 variables binarias activas. Se corren tres variantes en paralelo: LP con heurística, LP sin heurística, y vecino cercano de referencia.

Métrica	Con heurística	Sin heurística	Ref. Vecino
Distancia total	1852.02	1764.78	2006.73
Gap final	31.0%	29.2%	—
Variables tras prepro	5,922	10,000	—
Mejora vs. Vecino Ce	-7.7%	-12.0%	baseline

Tabla 3. Resultados Caso 3 — con y sin heurística de vecinos cercanos vs referencia (100 ciudades, mipgap=0.05, límite=60s).

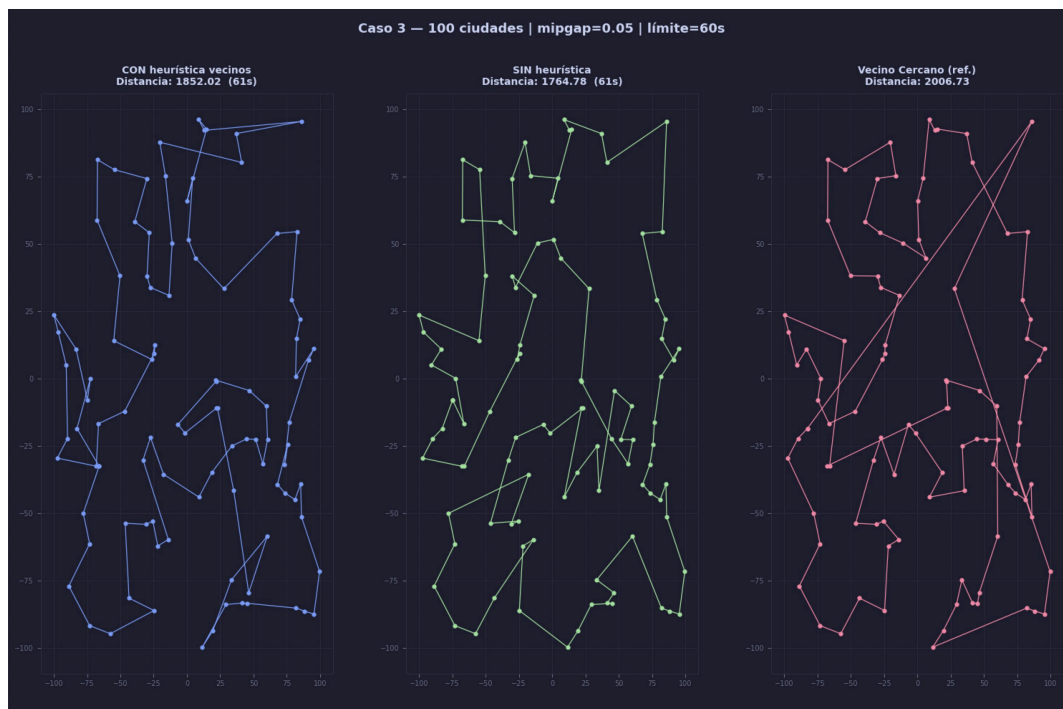


Figura 4. Vista del *tsp_viewer* — Caso 3: CON heurística vecinos (azul, 1852.02), SIN heurística (verde, 1764.78) y Vecino Cercano referencia (rosa, 2006.73). El LP sin heurística obtiene el mejor resultado.

¿Cuál es la diferencia entre los dos casos?

Resultado sorprendente: la heurística de vecinos cercanos empeoró la solución (1852.02 vs 1764.78). La reducción de variables no mejoró la solución porque las restricciones bloquearon arcos necesarios. Ambas versiones LP superaron al vecino cercano greedy (2006.73), pero el LP sin restricciones adicionales encontró una solución de mayor calidad aprovechando mejor el espacio de búsqueda.

¿Sirve esta heurística para cualquier caso?

No, y este caso lo demuestra empíricamente. La heurística asume que ninguna ciudad 'cercana' debería usar arcos largos, pero en distribuciones aleatorias siempre existen nodos que actúan como puentes geográficos entre regiones distantes. Al prohibirles usar arcos largos, el solver queda forzado a construir rutas subóptimas. Funcionaría mejor en instancias con clusters naturales bien definidos.

Conclusiones del TSP

Enfoque	Ciudades	Distancia	vs. Vecino Cercano	Tiempo
Vecino Cercano (greedy)	100	2006.73	baseline	< 1s
LP sin heurísticas — Caso 1	50	1096.57	-19.9%	30s 🕒
LP + límites — Caso	70	1473.05	—	40s 🕒
LP sin heurísticas — Caso 2	70	1651.85	—	40s 🕒
LP + vecinos — Caso 3	100	1852.02	-7.7%	60s 🕒
LP sin heurísticas — Caso 3	100	1764.78	-12.0%	60s 🕒

Tabla 4. Comparativa final de todos los enfoques TSP evaluados.

El TSP confirmó empíricamente su naturaleza NP-duro: 10 ciudades resuelven en segundos, 30 ya agotan el límite de 30s sistemáticamente. Las heurísticas son herramientas de doble filo: la de límites mejoró un 10.8% en el Caso 2, mientras que la de vecinos cercanos empeoró un 4.9% en el Caso 3. El LP sin heurísticas con tiempo suficiente demostró ser la estrategia más robusta para distribuciones aleatorias uniformes. La interfaz `tsp_viewer.py` facilitó el análisis cualitativo comparando visualmente las rutas lado a lado.

P3 — Algoritmos Genéticos

Se presenta un estudio comparativo de cinco configuraciones de un Algoritmo Genético (AG) aplicado a la generación evolutiva de la cadena de 17 caracteres "GA Workshop! USFQ". El AG debe evolucionar una población de cadenas aleatorias hasta reproducir exactamente el objetivo. Se analizan el efecto de la función de aptitud, la tasa de mutación, el tamaño de la población y operadores evolutivos avanzados. Los resultados muestran que la combinación de estos operadores

reduce el número de generaciones necesarias en un factor de 5–10× respecto a la configuración base.

Descripción del problema y ciclo evolutivo

Cromosoma: cadena de 17 caracteres (un gen por carácter). Alfabeto: 54 símbolos — a–z, A–Z, espacio y exclamación. Semilla aleatoria: 123 (reproducibilidad). Terminación: individuo igual al objetivo o 1,000 generaciones.

El AG sigue el ciclo estándar: (1) Evaluación — cálculo de aptitud para cada individuo; (2) Selección — elección de padres proporcional a la aptitud; (3) Cruce — combinación de dos padres para generar hijos; (4) Mutación — perturbación aleatoria de genes con probabilidad `mutation_rate`.

Caso 1 — Configuración DEFAULT

La función de aptitud cuenta el número de caracteres que coinciden exactamente en posición con el objetivo: $f(x) = \sum 1[x_i = o_i] \in \{0, \dots, 17\}$. Parámetros: población 100, `mutation_rate=0.01`, selección por ruleta proporcional, cruce de un punto.

La curva muestra crecimiento progresivo y estable desde las primeras generaciones, alcanzando el objetivo (17/17 coincidencias) alrededor de la generación 200–300. La selección proporcional garantiza que los caracteres correctos se propaguen sin destruir lo ya aprendido.

Caso 2 — Distancia de Hamming con Bug

La función `distance()` original acumulaba diferencias con signo entre códigos ASCII. Cuando una diferencia es positiva y otra negativa, ambas se cancelan: un individuo con errores compensados puede obtener distancia aparente cero, como si fuera idéntico al objetivo, eliminando por completo el gradiente de selección.

Resultado: la aptitud del mejor individuo oscila caóticamente entre 2–3 coincidencias durante las 1,000 generaciones sin mostrar mejora sostenida. Al eliminarse el gradiente evolutivo, el AG se comporta equivalente a búsqueda aleatoria.

Cuadro 2: Parámetros del Caso 2 (BY_DISTANCE con bug).

Parámetro	Valor
Población	100 individuos
Tasa de mutación	0.01 (1 %)
Iteraciones máx.	1 000
Función de aptitud	Distancia ASCII acumulada <i>sin abs()</i> (minimizar)
Selección de padres	Partición aleatoria; mejor de cada mitad
Cruce	Un punto

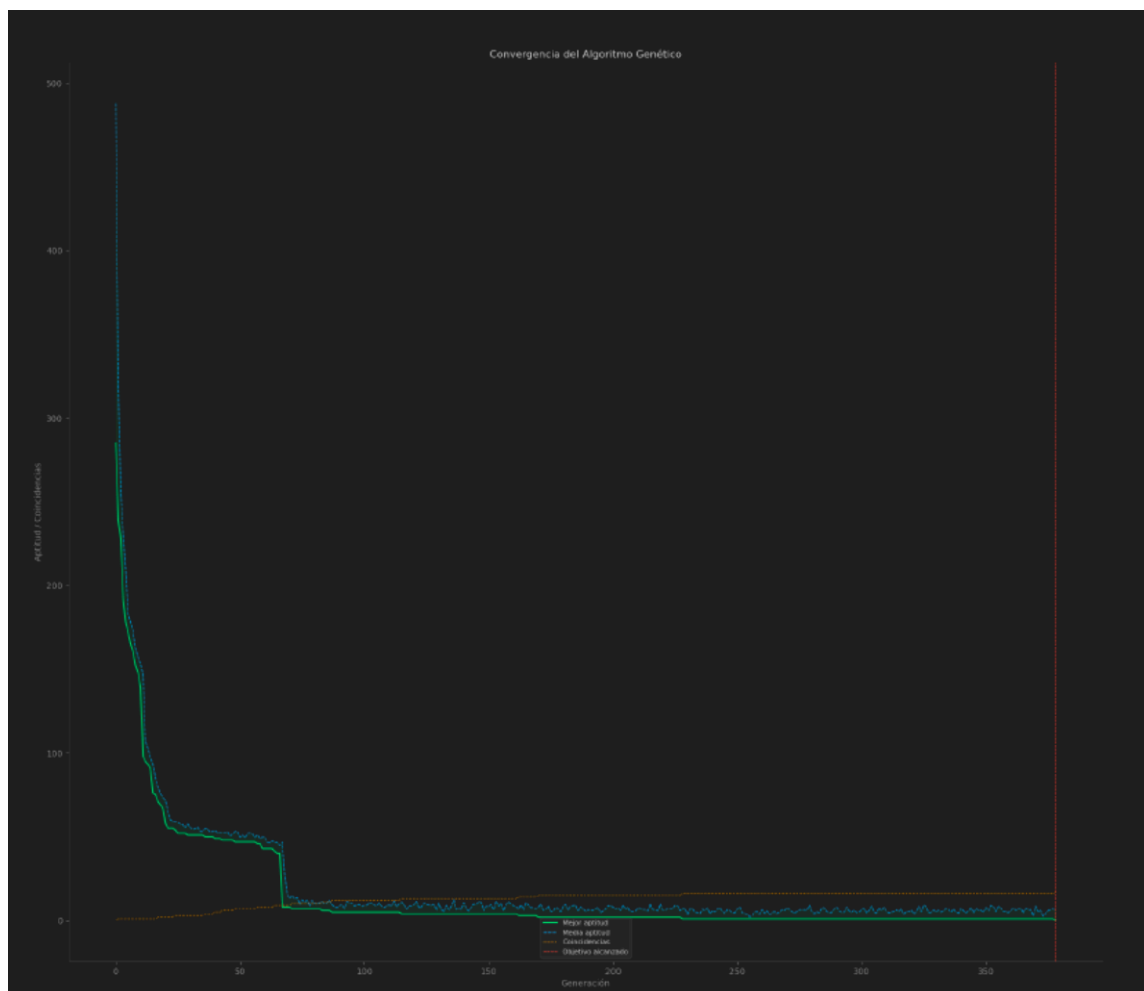


Figura 2: Caso 2 — Bug en distance ()

¿Por qué no finaliza como el Caso 1?

La función `distance()` sin `abs()` acumula diferencias con signo, lo que permite cancelaciones. La corrección aplica valor absoluto en cada término garantizando no negatividad y un único mínimo global cuando el individuo es idéntico al objetivo:

- Versión con bug: $\text{acc} += (e1 - e2)$ — las diferencias se cancelan
- Versión corregida: $\text{acc} = \text{sum}(\text{abs}(e1 - e2) \text{ for } e1, e2 \text{ in zip}(\text{list1}, \text{list2}))$



Figura 1: Caso 1 — DEFAULT

Caso 3 — Tasa de Mutación Alta (0.05)

Con $\text{mutation_rate}=0.05$ (5× mayor que el Caso 1), la tasa de mutación elevada introduce mayor diversidad genética y exploración del espacio de búsqueda, pero también destruye con más frecuencia genes correctos ya establecidos. La curva converge al objetivo con mayor varianza y oscilaciones, alcanzando 17/17 generalmente entre las generaciones 350–420.

El rango óptimo para este problema es 0.01–0.05. Por encima de 0.10, el AG degenera en búsqueda aleatoria porque las mutaciones destruyen el material genético útil más rápido de lo que la selección puede preservarlo.



Figura 3: Caso 3 — Mutación alta ($\text{mutation_rate} = 0.05$)

Caso 4 — Tamaño de Población Mayor (200)

Al duplicar el tamaño de la población (200 individuos) se aumenta significativamente la diversidad genética inicial, reduciendo el riesgo de convergencia prematura a óptimos locales. La curva de aptitud es más suave y alcanza el objetivo en aproximadamente 150–200 generaciones, un 30% más rápido que el Caso 1.

La ventaja tiene un costo computacional lineal: cada generación procesa el doble de individuos, lo que representa un equilibrio favorable entre velocidad de convergencia y tiempo de cómputo.

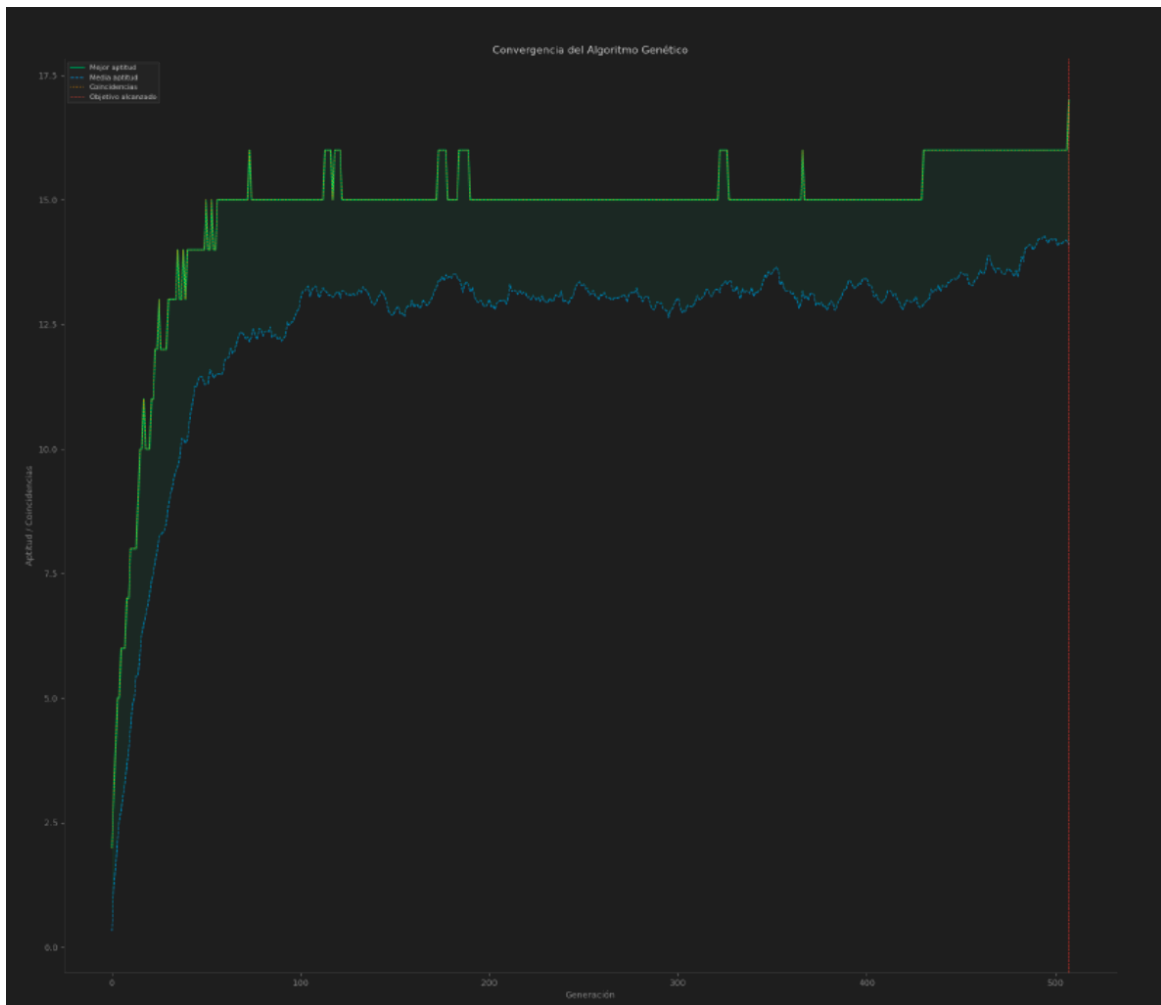


Figura 4: Caso 4 — Población 200

Caso 5 — Combinación Optimizada (IMPROVED)

Parámetro	Valor
Población	200
Tasa de mutación	0.05
Selección de padres	Torneo (k = 3)
Cruce	Dos puntos
Elitismo	Top 10% pasa intacto a la siguiente generación

Tabla 5. Parámetros del Caso 5 — IMPROVED.

La combinación de elitismo, selección por torneo, cruce de dos puntos, mayor población y tasa de mutación 0.05 produce la convergencia más rápida: 17/17 coincidencias en aproximadamente 25–60 generaciones, una reducción de 5–10× respecto al Caso 1.

- Elitismo (top 10%): garantiza que la mejor aptitud nunca retrocede — el impacto de mayor mejora individual.
- Torneo ($k=3$): escoge al mejor de tres candidatos aleatorios, aumentando la presión selectiva sin monopolio.
- Cruce de dos puntos: preserva bloques correctos en ambos extremos del cromosoma, reduciendo el sesgo posicional.

Cuadro 5: Parámetros del Caso 5 (IMPROVED, combinación de todas las mejoras).

Parámetro	Valor
Población	200
Tasa de mutación	0.05
Selección de padres	Torneo ($k = 3$)
Cruce	Dos puntos
Elitismo	Top 10 % pasa intacto a la siguiente generación

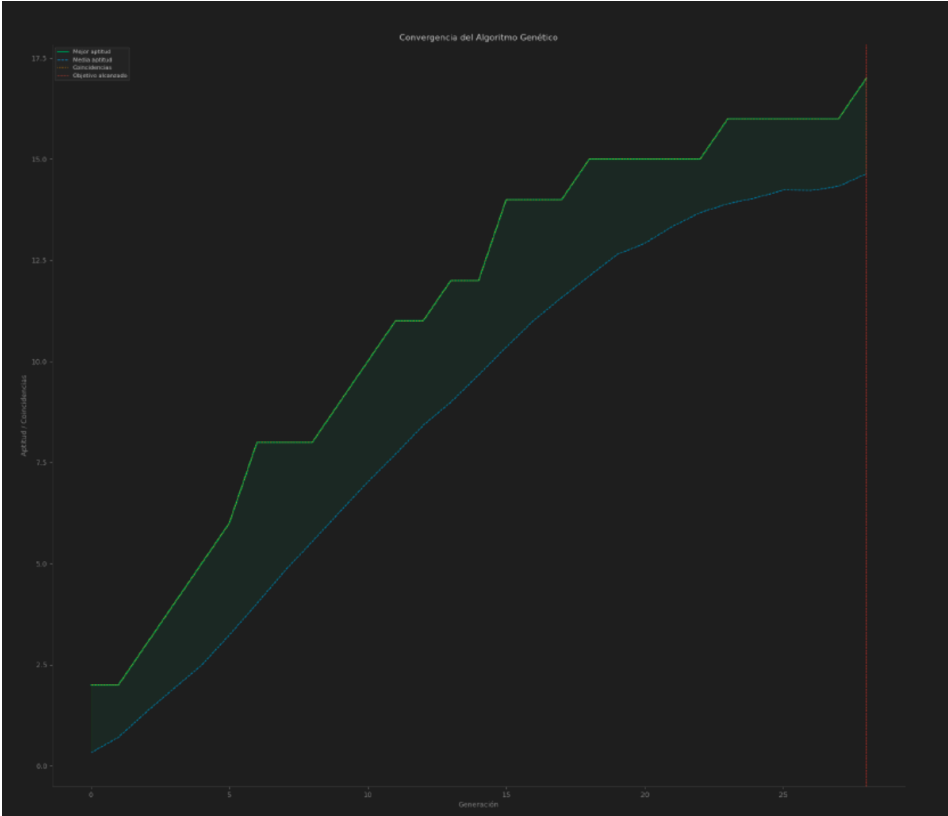


Figura 5: Caso 5 — IMPROVED

Análisis Comparativo

Caso	Pob.	Mut.	Selección	Cruce	Convergencia
1 —	100	0.01	Ruleta	1 punto	~200–400 gen.
2 — Bug distance()	100	0.01	Partición	1 punto	No converge
3 — Mutación	100	0.05	Ruleta	1 punto	~350–420 gen.
4 — Población	200	0.01	Ruleta	1 punto	~150–200 gen.
5 — IMPROVE	200	0.05	Torneo k=3	2 puntos	~25–60 gen.

Tabla 6. Comparativa de los cinco casos de estudio del Algoritmo Genético.

Conclusiones del Algoritmo Genético

- una función defectuosa (Caso 2) elimina el gradiente evolutivo y convierte el AG en búsqueda aleatoria, independientemente de los demás parámetros. La función de aptitud es crítica:
- cambiar selección, cruce y añadir elitismo (Caso 5) reduce las generaciones en 5–10× sin modificar la complejidad algorítmica. La arquitectura de operadores supera el ajuste de parámetros:
- garantiza que la mejor aptitud nunca retrocede, acelerando la convergencia de forma robusta y estable. El elitismo es la mejora de mayor impacto individual:
- mayor población (Caso 4) aporta diversidad con costo lineal; mutación moderada (0.01–0.05) equilibra exploración y explotación. Población y mutación son complementarios:
- converge en 25–60 generaciones vs 200–400 del Caso 1, demostrando que el diseño integrado de operadores es la estrategia más efectiva. El Caso 5 valida la sinergia de todas las mejoras:

Conclusión General

El desarrollo de este taller permitió abordar tres tipos distintos de problemas de inteligencia artificial: aprendizaje no supervisado para descubrimiento de patrones, optimización combinatoria NP-duro con modelos LP y heurísticas, y algoritmos evolutivos para búsqueda en espacios de cadenas.

En el aprendizaje no supervisado, la naturaleza sintética del dataset limitó la profundidad de los patrones encontrados, pero el análisis multivariable con Isolation Forest demostró ser significativamente más revelador que el análisis univariable, identificando perfiles estudiantiles atípicos con potencial de intervención educativa. En el TSP, se confirmó empíricamente la complejidad NP-duro: la calidad de la solución depende del tiempo disponible y de las heurísticas aplicadas. Las heurísticas son herramientas de doble filo — pueden acelerar la convergencia o empeorar los resultados según la geometría del problema. La interfaz gráfica `tsp_viewer.py` aportó valor analítico al permitir comparar visualmente las rutas de cada configuración.

En los Algoritmos Genéticos, el Caso 5 demostró que el diseño integrado de operadores (elitismo, torneo, cruce de dos puntos) supera ampliamente el ajuste individual de parámetros, logrando convergencia en 25–60 generaciones vs 200–400 de la configuración base — una mejora de 5–10×.

Transversalmente, los tres problemas ilustran un principio fundamental: la representación correcta del problema y la selección adecuada del algoritmo son tan importantes como la implementación técnica. La visualización interactiva y las interfaces gráficas facilitaron significativamente la comprensión de algoritmos complejos y la interpretación de los resultados.